



FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Tarea 2

Introducción a la Ciencia de Datos

Edición 2026

Integrantes:

Tomás Alejo Spoturno Gonzalez, 5.165.789-6

Nicolás Buero, 5.000.105-4

Universidad de la República
Facultad de Ingeniería

Junio 2026

Índice

1. Introducción	2
2. Parte 1: Dataset y representación numérica de texto	2
2.1. Preparación del dataset	2
2.2. Verificación del balance de clases	3
2.3. Representación Bag of Words	3
2.4. Representación TF-IDF	5
2.5. Visualización PCA sobre TF-IDF	5
3. Parte 2: Entrenamiento y evaluación de modelos	12
3.1. Multinomial Naive Bayes	12
3.2. Validación cruzada y búsqueda de hiperparámetros	13

1. Introducción

Este informe documenta la resolución de la Tarea 2 del curso *Introducción a la Ciencia de Datos* (edición 2026). La tarea es continuación directa de la Tarea 1 y utiliza el mismo dataset: un subconjunto muestreado del corpus abierto *All the News 2.1 - Component One*, que contiene artículos de noticias publicados en distintos medios de prensa. El objetivo de esta entrega es **construir y evaluar clasificadores automáticos de texto** que identifiquen a qué medio pertenece cada artículo, partiendo de representaciones numéricas del texto.

El trabajo se estructura en dos partes. La Parte 1 cubre la preparación de los datos, la representación numérica del texto mediante *Bag of Words* y TF-IDF, y una exploración visual con PCA. La Parte 2 cubre el entrenamiento, la validación y la comparación de modelos de clasificación.

El código que genera los resultados está en el notebook `Tarea_2/2026_tarea2.ipynb` del repositorio público <https://github.com/Nicolasvb23/intro-cd>.

2. Parte 1: Dataset y representación numérica de texto

2.1. Preparación del dataset

El proceso de limpieza de datos y la función `clean_text` son los mismos que en la Tarea 1, a los que se remite para el detalle. En resumen, sobre el dataset original de 30.213 artículos se eliminaron 141 filas con `publication` nula, 1.176 con `article` nulo o placeholder, y 13 duplicados exactos, resultando en **29.024 artículos limpios**. La función `clean_text` aplica los mismos pasos que en la entrega anterior (eliminación de cabecera, minúsculas, URLs, puntuación, números aislados y normalización de espacios), sin agregar filtrado de *stopwords* ni identificadores de medios en esta etapa, ya que esas decisiones se toman a nivel del vectorizador y son parte del análisis de la Sección 2.5.

Selección de los tres medios principales

El análisis de esta tarea se restringe a los **tres medios con mayor cantidad de artículos** en `df_clean`. Los recuentos se muestran en la Tabla 1.

Cuadro 1: Top 3 medios de prensa por cantidad de artículos en el dataset limpio.

Medio	Artículos	% del total (top 3)
Reuters	9.266	63,2 %
The New York Times	2.805	19,1 %
CNBC	2.578	17,6 %
Total top 3	14.649	100 %

Los tres medios coinciden con los tres primeros del top 5 de la Tarea 1. El dataset resultante presenta un **desbalance pronunciado**: *Reuters* acumula el 63,2 % de los artículos, frente al

19,1 % de *NYT* y el 17,6 % de *CNBC*. Esto tiene implicancias sobre las métricas de evaluación que se discuten en la Parte 2.

Partición train/test

El dataset de los tres medios se dividió en un conjunto de **entrenamiento (70 %)** y un conjunto de **test (30 %)**, utilizando muestreo estratificado por medio de prensa (`stratify=y`). La estratificación garantiza que la proporción de artículos de cada medio sea la misma en ambos conjuntos. Se fijó `random_state=23` para reproducibilidad.

2.2. Verificación del balance de clases

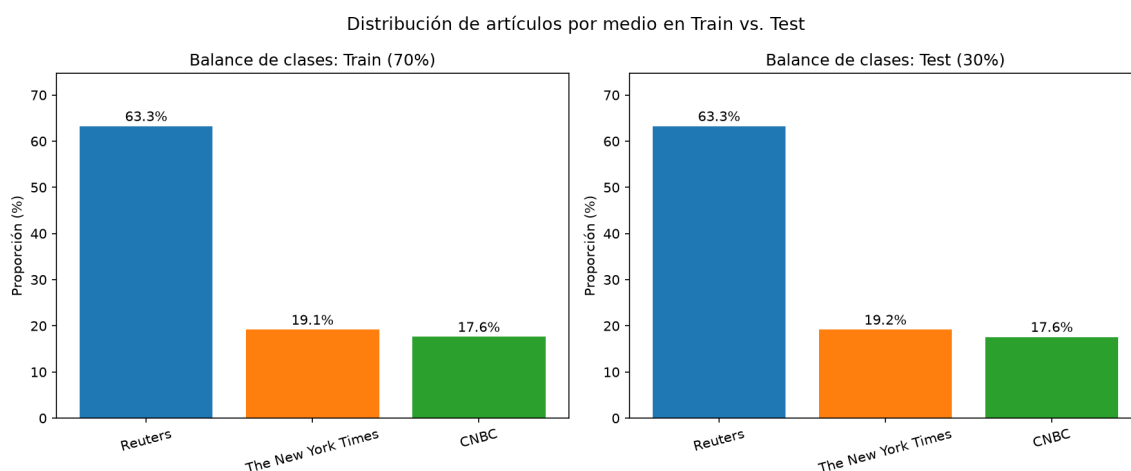


Figura 1: Proporción de artículos de cada medio en los conjuntos de entrenamiento (izquierda) y test (derecha). La estratificación garantiza distribuciones idénticas en ambos conjuntos.

La Figura 1 confirma que las proporciones son idénticas en train y test, como se espera de un split estratificado. El desbalance del dataset se preserva de forma exacta: aproximadamente 62 % de los artículos corresponden a *Reuters*, lo que implica que un clasificador trivial que siempre prediga “Reuters” alcanzaría un *accuracy* de ese orden sin aprender nada del contenido. Este punto se retoma en la Parte 2 al discutir las limitaciones del *accuracy* como única métrica.

2.3. Representación Bag of Words

Cómo funciona

La representación *Bag of Words* (BoW) convierte cada documento en un vector numérico de la siguiente manera: se construye un **vocabulario** fijo con todas las palabras distintas que aparecen en el corpus de entrenamiento, y cada documento se representa como un vector de tamaño igual al vocabulario donde la posición j contiene la **cantidad de veces** que la palabra j aparece en ese documento. El orden de las palabras dentro del texto se ignora completamente.

Por ejemplo, dado el siguiente corpus de tres frases:

Doc	Texto
d_1	"market falls on trade fears"
d_2	"trump trade war fears"
d_3	"market reacts to trump"

El vocabulario resultante es {falls, fears, market, on, reacts, to, trade, trump, war} y la matriz BoW es:

	falls	fears	market	on	reacts	to	trade	trump	war
d_1	1	1	1	1	0	0	1	0	0
d_2	0	1	0	0	0	0	1	1	1
d_3	0	0	1	0	1	1	0	1	0

Cada fila es el vector numérico que representa al documento. Notar que "market falls" y "falls market" producirían exactamente el mismo vector: el orden se pierde.

Tamaño de la matriz

La matriz BoW ajustada sobre el conjunto de entrenamiento tiene dimensiones [n_docs × n_palabras], donde:

- **Filas:** 10.254 artículos en el conjunto de entrenamiento.
- **Columnas:** 84.358 palabras distintas en el vocabulario aprendido sobre el train.

Por qué es una matriz *sparse*

Si se almacenara como una matriz densa (todos los valores en memoria), la matriz BoW ocuparía aproximadamente:

$$10,254 \times 84,358 \times 8 \text{ bytes} \approx 6,920 \text{ MB}$$

donde los 8 bytes por elemento corresponden al tipo `float64` que NumPy usa por defecto. Esto sería inmanejable incluso con este dataset reducido. Sin embargo, la gran mayoría de las entradas son cero: la densidad de la matriz es del 0,247 %, lo que significa que más del 99 % de sus valores son cero. Ninguna nota usa más de unos cientos de palabras distintas, mientras que el vocabulario total tiene decenas de miles.

Las matrices *sparse* en formato CSR almacenan únicamente los valores no-nulos y sus posiciones mediante tres arrays: los valores (`float64`, 8 bytes cada uno), los índices de columna (`int32`, 4 bytes cada uno) y los punteros de fila (`int32`, despreciable). Con $\approx 2,14$ millones de entradas no-nulas (densidad × tamaño total), el costo es:

$$2.136.567 \times (8 + 4) \text{ bytes} \approx 25,7 \text{ MB}$$

una reducción de 6.920 MB a tan solo 25,7 MB.

Si en lugar de 14.000 artículos tuviéramos el dataset completo de 1,8 millones de artículos del corpus original y un vocabulario de cientos de miles de palabras, la matriz densa sería completamente inviable en cualquier equipo de cómputo estándar.

2.4. Representación TF-IDF

Qué es un n-grama

Un **n-grama** es una secuencia contigua de n tokens (palabras o caracteres). Un unigrama ($n = 1$) es una palabra individual; un bigrama ($n = 2$) es un par de palabras consecutivas. Por ejemplo, a partir del texto “*the new york times*” se extraen los bigramas “*the new*”, “*new york*” y “*york times*”. Incluir bigramas en el vocabulario permite capturar expresiones compuestas y contexto local que los unigramas no pueden representar. El parámetro `ngram_range=(1,2)` en el vectorizador combina unigramas y bigramas en el mismo vocabulario, aunque a costa de aumentar dramáticamente el número de columnas.

La transformación TF-IDF

El principal problema de BoW es que las palabras muy frecuentes en todo el corpus (artículos, preposiciones, conjunciones) dominan los vectores aunque no aporten información discriminante entre medios. TF-IDF (*Term Frequency - Inverse Document Frequency*) corrige esto ponderando cada conteo por la siguiente expresión:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

donde $\text{TF}(t, d)$ es la frecuencia normalizada del término t en el documento d , e $\text{IDF}(t) = \log\left(\frac{N+1}{n_t+1}\right) + 1$ con N el número total de documentos y n_t la cantidad de documentos que contienen t . La transformación tiene dos efectos:

- **Penaliza términos ubicuos:** una palabra que aparece en todos o casi todos los documentos tiene $\text{IDF} \approx 0$, por lo que su peso TF-IDF cae a cero o próximo a cero, independientemente de cuántas veces aparezca en un documento dado.
- **Premia términos discriminantes:** una palabra rara que aparece mucho en un documento particular y poco en el resto recibe un peso alto.

El vectorizador se ajusta únicamente sobre el conjunto de entrenamiento. El vocabulario y los valores IDF aprendidos se aplican luego sobre el conjunto de test, sin recalcularse nada sobre él. Esto simula correctamente el escenario real donde el modelo no tiene acceso a los datos de evaluación al momento de entrenarse.

2.5. Visualización PCA sobre TF-IDF

Consideraciones de implementación

El análisis de componentes principales (PCA) requiere materializar la matriz TF-IDF como densa para poder centrarla. Con el vocabulario completo de 84.358 palabras y 10.254 documentos de

entrenamiento, esto supone aproximadamente 6,9 GB en RAM, lo que es viable dado el equipo disponible. PCA es el método solicitado en el enunciado de la tarea para esta visualización.

Resultados con la configuración base

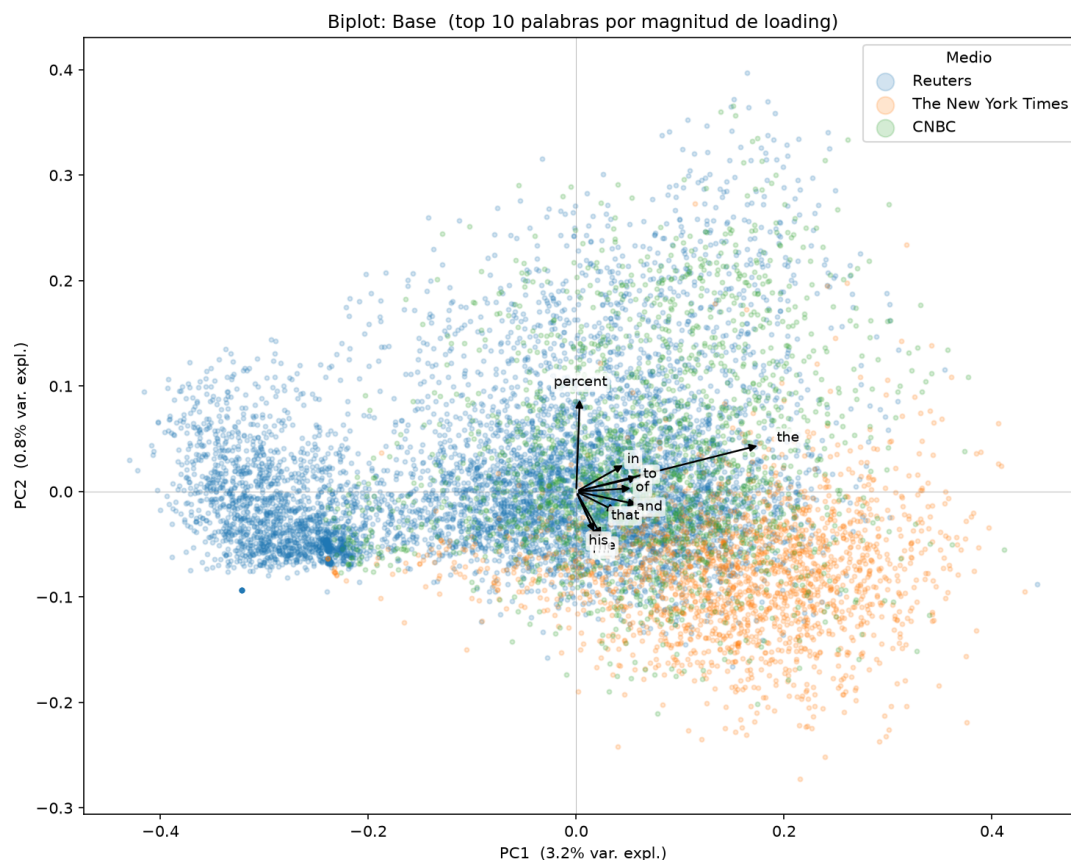


Figura 2: Biplot sobre TF-IDF base: puntos coloreados por medio de prensa y flechas con las 15 palabras de mayor loading en el plano PC1-PC2.

Cómo leer un biplot

En un biplot, cada flecha representa un término del vocabulario. Su **longitud** es proporcional a la magnitud del loading en el plano mostrado: una flecha larga indica que ese término contribuye significativamente a las componentes graficadas. Su **dirección** indica hacia qué zona del espacio “apunta” ese término: los documentos ubicados en esa dirección tienden a tener valores altos para ese feature. El **ángulo entre dos flechas** aproxima la correlación entre ambos términos: flechas paralelas indican covarianza positiva, flechas opuestas indican covarianza negativa, y flechas perpendiculares indican términos prácticamente no correlacionados.

Sin embargo, toda interpretación de un biplot es válida **solo dentro del subespacio proyectado**. Si las primeras dos componentes explican una fracción pequeña de la varianza total, los ángulos y direcciones visibles pueden diferir considerablemente de las relaciones reales en el espacio completo. En este caso, PC1 y PC2 explican apenas el 4,1 % de la varianza: las observaciones deben tomarse

como indicios cualitativos de las direcciones más prominentes, no como afirmaciones precisas sobre la estructura global de los datos.

La Figura 2 muestra el conjunto de entrenamiento proyectado sobre las dos primeras componentes principales. *Reuters* forma una nube claramente separada del resto en el extremo **negativo** de PC1, mientras que *NYT* y *CNBC* se concentran en el lado **positivo** con mayor superposición entre sí. Las dos primeras componentes explican solo el 4,1 % de la varianza total: los vectores TF-IDF viven en un espacio de alta dimensionalidad y la proyección bidimensional pierde la mayoría de la información.

El término con flecha más larga es “*the*”, paralela a PC1 apuntando hacia *NYT/CNBC*: los artículos de esos medios tienen mayor frecuencia relativa de “*the*” que el estilo telegráfico de *Reuters*. “*percent*” apunta hacia arriba (PC2 positivo), hacia la zona de *NYT* y *CNBC*, reflejando cobertura de noticias numéricas. “*million*” apunta en dirección opuesta (arriba-izquierda, hacia *Reuters*), consistente con el estilo del wire financiero. “*text*” apunta directamente hacia *Reuters*, posiblemente como artefacto de boilerplate del medio. El resto de los términos visibles (“*in*”, “*to*”, “*of*”, “*and*”, “*that*”) son *stop words* con flechas muy cortas y sin dirección clara: IDF los penaliza fuertemente pero no los elimina por completo gracias al suavizado aditivo de la fórmula.

Comparación de configuraciones

Se analiza el efecto de tres modificaciones al vectorizador TF-IDF. Para cada configuración se muestra el biplot correspondiente.

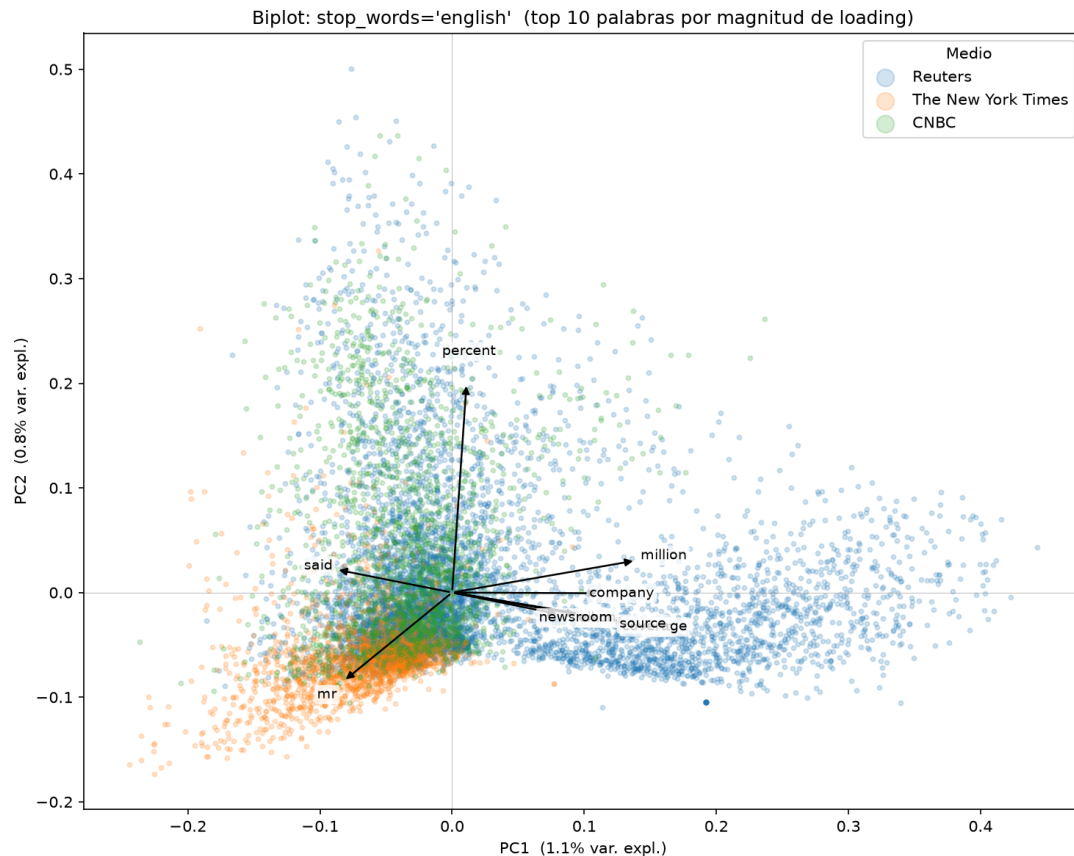


Figura 3: Biplot con `stop_words='english'`: al eliminar palabras funcionales, los loadings quedan dominados por términos de contenido. PC1 explica 1,1 % de la varianza.

a) Filtrado de *stop words* (`stop_words='english'`). Al eliminar las palabras funcionales del inglés (artículos, preposiciones, conjunciones), palabras como “*the*”, “*of*” o “*in*” desaparecen del vocabulario y quedan expuestos términos de contenido. La Figura 3 revela una separación temática clara entre dos grupos de flechas con direcciones opuestas.

Hacia el lado de *Reuters* aparecen “*share*”, “*yuan*”, “*million*” y “*company*”: vocabulario típico del wire financiero internacional, que cubre cotizaciones bursátiles, divisas asiáticas y resultados corporativos. Hacia *NYT* y *CNBC* apunta “*trump*”, reflejo de la cobertura política estadounidense que domina esos medios. Que estas flechas sean aproximadamente opuestas entre sí indica correlación negativa en el plano proyectado: los artículos que usan mucho “*yuan*” o “*share*” tienden a no usar “*trump*”, y viceversa.

Esta separación editorial no era visible en la configuración base porque las palabras funcionales dominaban los loadings. Contra lo intuitivo, PC1 cae de 3,2 % a 1,1 %: al eliminar los términos más frecuentes se reduce la concentración de varianza en la primera dirección, distribuyéndola entre más componentes.

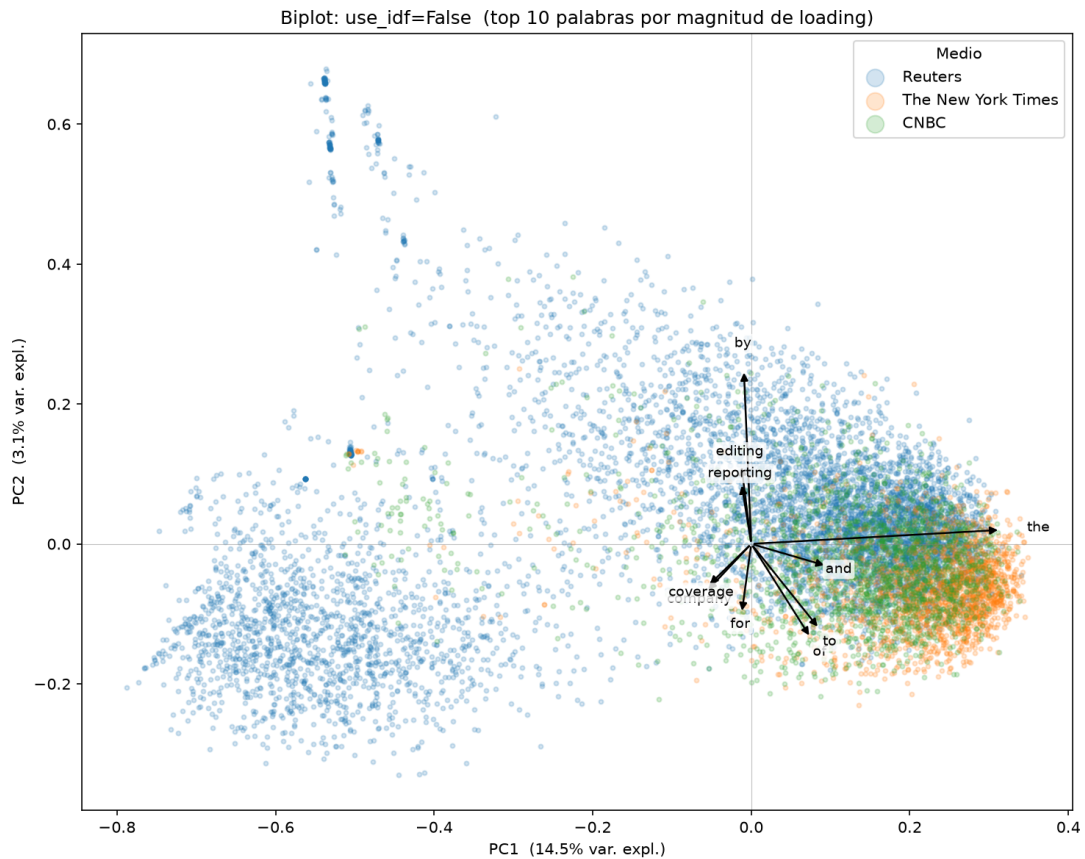


Figura 4: Biplot con `use_idf=False`: una flecha larga domina PC1, correspondiente a *“the”*. PC1 explica 14,5 % de la varianza.

b) Sin IDF (`use_idf=False`). Con IDF desactivado, las palabras muy comunes recuperan el peso que el factor inverso les quitaba. El biplot (Figura 4) ilustra el problema con claridad: aparece una única flecha dominante, notablemente más larga que el resto, correspondiente a *“the”*. PC1 salta a 14,5 % de varianza explicada (frente al 3,2 % de la configuración base) porque ahora la primera componente captura principalmente variación en la frecuencia de ese artículo, que es equidistribuida entre los tres medios y no aporta discriminación. Esto ilustra concretamente por qué el factor IDF es fundamental en clasificación de texto.

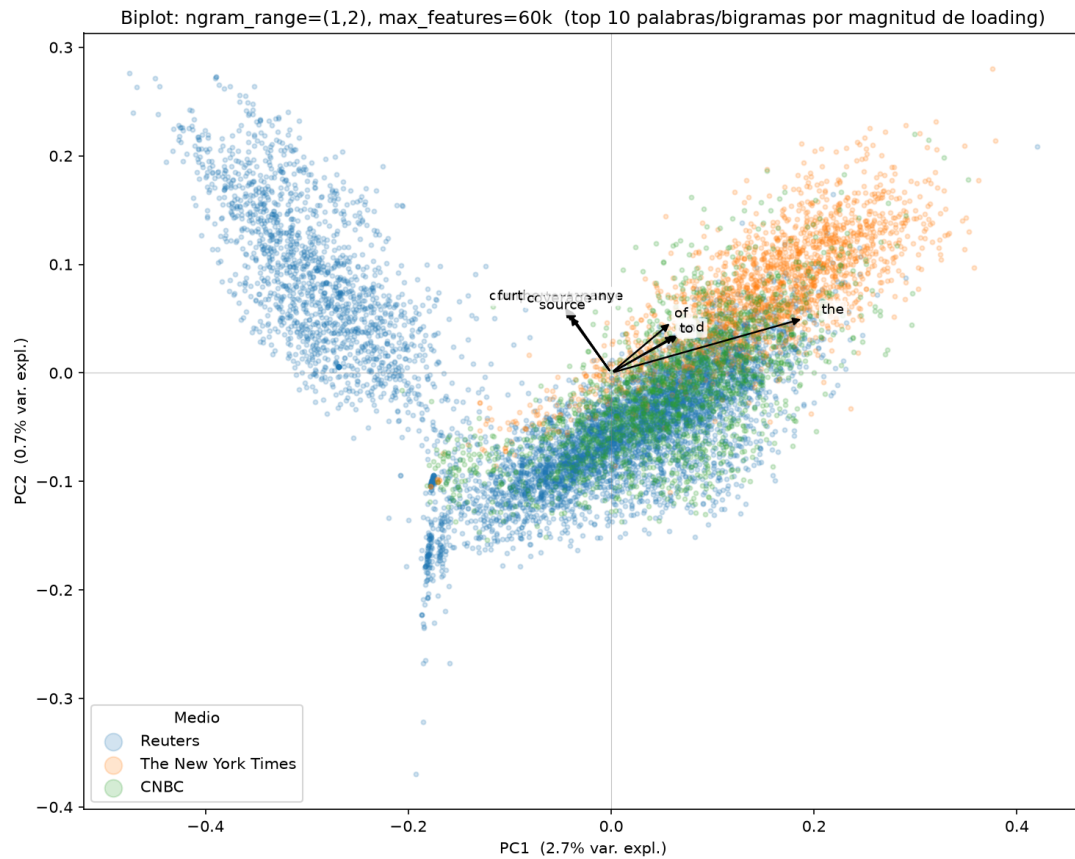


Figura 5: Biplot con `ngram_range=(1,2)` y `max_features=60.000`: el vocabulario incluye secuencias de dos palabras consecutivas. PC1 explica 3,0 % de la varianza.

c) Bigramas (`ngram_range=(1,2)`). Al agregar bigramas el vocabulario sin límite incluiría unigramas más todas las combinaciones de dos palabras consecutivas, lo que lo haría crecer hasta cientos de miles de términos. Para hacer viable la densificación se acotó a `max_features=60.000`, que es incluso menor que el vocabulario base de 84.358 unigramas: el límite selecciona los 60.000 términos más frecuentes, que en su mayoría siguen siendo unigramas.

Esto se refleja en el biplot (Figura 5): las flechas no muestran una dirección clara ni una separación definida entre medios, y los términos de mayor loading son en su mayoría unigramas, no bigramas. La varianza explicada por PC1 (3,0 %) es prácticamente igual a la base (3,2 %), lo que confirma que agregar bigramas con este límite no aporta información discriminante adicional en las primeras dos componentes.

Varianza explicada

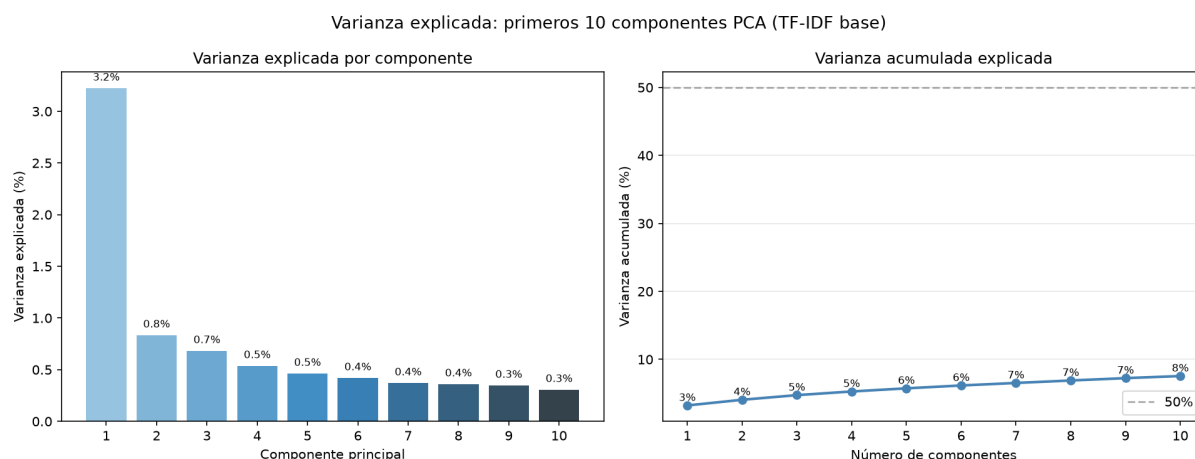


Figura 6: Varianza explicada por cada componente principal (izquierda) y varianza acumulada (derecha) para los primeros 10 componentes sobre la configuración TF-IDF base.

La Figura 6 muestra que la varianza explicada decrece rápidamente con el número de componentes y que incluso con 10 componentes la varianza acumulada es relativamente baja. Esto refleja la **alta dimensionalidad intrínseca del texto**: la información está repartida entre miles de términos, y ninguna dirección captura una fracción grande de la varianza total.

Origen de la separación observada

El análisis de los biplots permite identificar dos fuentes de separación entre los medios: diferencias de **estilo editorial** y diferencias de **contenido temático**.

Reuters se separa del resto a lo largo de PC1 por ambas razones a la vez. Por un lado, “the” es el término de mayor loading en el biplot base, apuntando hacia *NYT/CNBC*, lo que indica que los artículos de esos medios tienen mayor frecuencia relativa de ese término que los de *Reuters*. Por otro lado, su cobertura temática de wire financiero internacional genera vocabulario específico (“yuan”, “share”, “million”, “company”) que apunta hacia el lado de *Reuters* en el biplot sin stop words. El término “text” apuntando también hacia *Reuters* sugiere además la presencia de algún boilerplate propio del medio que sobrevivió a la limpieza.

NYT y *CNBC* no se separan bien entre sí con solo 2 componentes. Ambos quedan del mismo lado de PC1 por compartir cobertura político-económica estadounidense: “trump” apunta hacia ambos, y “market” también. La diferencia entre estos dos medios requiere componentes más allá de las dos primeras, que en conjunto explican apenas el 4,1 % de la varianza.

En definitiva, con 2 componentes PCA **no es posible separar los tres medios de forma limpia**. *Reuters* sí queda claramente diferenciado, pero *NYT* y *CNBC* se superponen en el plano proyectado. La separación observada es una combinación de estilo (frecuencia de palabras funcionales, longitud de artículos) y contenido temático (finanzas internacionales vs. política estadounidense), sin que sea posible atribuirla a una sola causa.

3. Parte 2: Entrenamiento y evaluación de modelos

3.1. Multinomial Naive Bayes

Cómo funciona

El clasificador *Multinomial Naive Bayes* (MNB) estima la probabilidad de que un documento d pertenezca a una clase c mediante el teorema de Bayes:

$$P(c | d) \propto P(c) \prod_{t \in d} P(t | c)^{f_{t,d}}$$

donde $P(c)$ es la probabilidad a priori de la clase (proporcional a su frecuencia en el training), $P(t | c)$ es la probabilidad del término t en los documentos de clase c , y $f_{t,d}$ es la frecuencia del término en el documento. La suposición “naive” es que los términos son condicionalmente independientes dada la clase, lo que raramente se cumple en la práctica pero simplifica el cálculo y funciona sorprendentemente bien en texto. El parámetro α controla el suavizado de Laplace, que evita probabilidades nulas para términos no vistos en el training.

Resultados

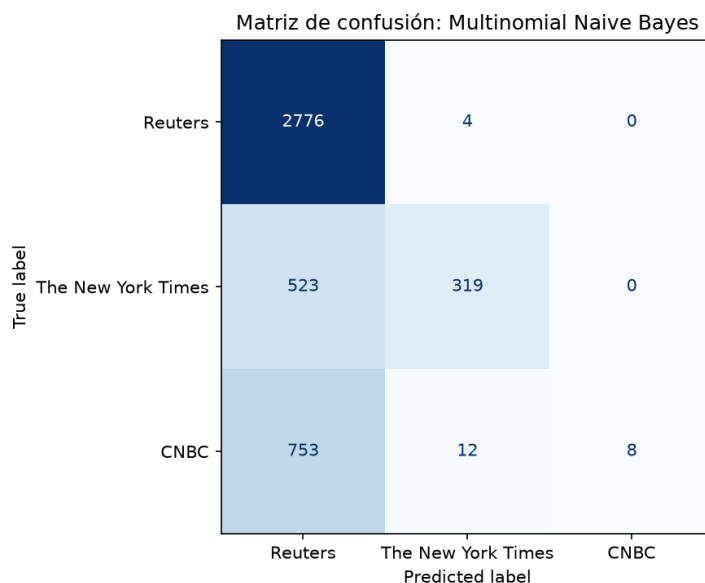


Figura 7: Matriz de confusión del clasificador Multinomial Naive Bayes sobre el conjunto de test.

El modelo alcanza un *accuracy* del **70,6 %** sobre el conjunto de test. La Tabla 2 desglosa las métricas por medio.

Cuadro 2: Precision, recall y F1-score del clasificador MNB por medio de prensa.

Medio	Precision	Recall	F1	Support
Reuters	0,69	1,00	0,81	2.780
The New York Times	0,95	0,38	0,54	842
CNBC	1,00	0,01	0,02	773
Macro avg	0,88	0,46	0,46	4.395
Weighted avg	0,79	0,71	0,62	4.395

Análisis

Los resultados revelan un problema clásico de datos desbalanceados. El modelo clasifica prácticamente todo como *Reuters* (recall=1,00), que es la clase mayoritaria con el 63,3 % de los datos. Al usar las frecuencias de clase como probabilidades a priori, MNB aprende que apostar a *Reuters* es rentable: ese sesgo le garantiza acertar todos los artículos de esa fuente. Como consecuencia, *CNBC* queda casi invisible para el modelo (recall=0,01), e *NYT* se identifica correctamente solo en el 38 % de los casos.

La relación con la matriz de confusión (Figura 7) es directa: la columna de *Reuters* concentra la mayoría de las predicciones, y las filas de *NYT* y *CNBC* muestran que la mayor parte de esos artículos es enviada erróneamente a *Reuters*.

Limitaciones del *accuracy*

El *accuracy* del 70,6 % es engañoso: un clasificador trivial que siempre prediga *Reuters* alcanzaría el 63,3 % sin aprender nada del contenido. El modelo MNB base solo mejora 7 puntos sobre esa línea de base. Si el desbalance fuera aún mayor (por ejemplo, 95 % de una clase), el *accuracy* podría superar el 95 % con un clasificador que ignora completamente las clases minoritarias. Por eso es fundamental complementar el *accuracy* con métricas por clase como *precision*, *recall* y F1, que exponen el comportamiento del modelo en cada clase individualmente.

3.2. Validación cruzada y búsqueda de hiperparámetros

Validación cruzada

La validación cruzada de k folds divide el conjunto de entrenamiento en k partes iguales. En cada iteración, $k - 1$ partes se usan para entrenar y la parte restante para evaluar; esto se repite k veces rotando la parte de evaluación. El resultado final es el promedio de las k métricas obtenidas. La ventaja frente a una sola partición train/validación es que todos los datos participan tanto en entrenamiento como en evaluación, obteniendo una estimación más robusta y con menor varianza. El conjunto de test nunca se toca durante este proceso.

Búsqueda de hiperparámetros

Se realizó una búsqueda exhaustiva (*GridSearchCV*) sobre el parámetro α del suavizado de Laplace, usando validación cruzada de 5 folds y F1-macro como métrica de selección. Los valores explorados fueron $\alpha \in \{0,01; 0,05; 0,1; 0,5; 1,0; 2,0; 5,0; 10,0\}$.

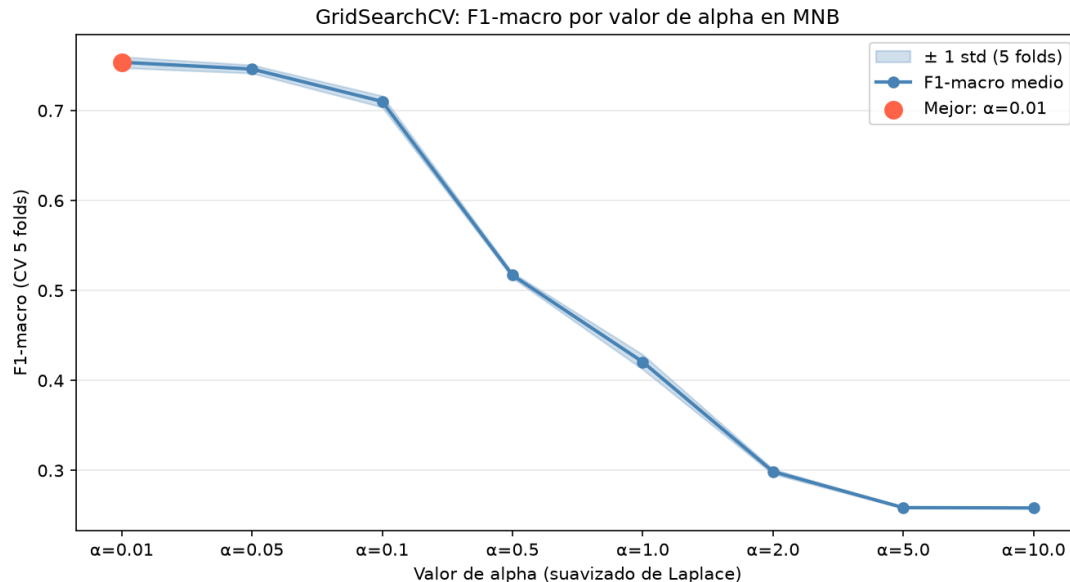


Figura 8: F1-macro medio (5 folds) en función del valor de α . La banda sombreada indica ± 1 desviación estándar entre folds. El punto rojo marca el mejor valor encontrado.

La Figura 8 muestra una caída monótona del F1-macro a medida que α aumenta: el mejor valor es $\alpha = 0,01$ con F1-macro = 0,754, mientras que $\alpha = 10$ colapsa a 0,258. Un α grande suaviza excesivamente las probabilidades de los términos, acercando las distribuciones de todas las clases entre sí y destruyendo la información discriminante. El mejor $\alpha = 0,01$ indica que con TF-IDF el modelo se beneficia de un suavizado mínimo.

Se optó por un gráfico de línea con banda de error en lugar del gráfico de violín sugerido en el enunciado porque la varianza entre folds es muy baja (desviación estándar $\approx 0,006$) frente al rango total de variación entre valores de α ($\approx 0,50$). Con solo 5 puntos por violín, la estimación de densidad produce formas planas e ilegibles; el gráfico de línea transmite la misma información de forma más clara.